Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Variables, Affectation, Structures Conditionnelles et Boucles Itératives

Synthèse exécutive : Ce cours pose les fondations fondamentales de la programmation et de l'algorithmique en BUT Informatique. Il explique comment un ordinateur mémorise des informations via des variables en mémoire vive (RAM) et comment il prend des décisions logiques à l'aide de structures conditionnelles (Si / Sinon) et d'itérations répétitives (Boucles Pour et TantQue). L'objectif est d'apprendre à traduire un problème du monde réel en une suite d'instructions formelles, séquentielles et non ambiguës exécutables par une machine.

1. Mémorisation et Manipulation des Données : Les Variables

En informatique, une variable ne doit pas être confondue avec une variable mathématique. Il s'agit d'une 'boîte' ou d'un emplacement étiqueté dans la mémoire vive (RAM) de l'ordinateur. Une variable possède quatre caractéristiques indissociables : un nom (identificateur), un type de donnée (qui définit la taille mémoire et le format d'encodage), une valeur (le contenu exact à un instant t) et une adresse mémoire physique. L'opération d'affectation consiste à évaluer une expression située à droite du symbole d'affectation et à ranger le résultat obtenu dans la case mémoire identifiée à gauche. L'ancienne valeur contenue dans la case est alors définitivement écrasée.

- Variable : Espace mémoire nommé réservé en RAM pour stocker une valeur modifiable au cours de l'exécution.
- Typage : Diction formel de la nature de la donnée (Entier, Réel, Chaîne de caractères, Booléen) déterminant sa taille en octets et ses opérations autorisées.
- Affectation : Opération fondamentale d'écriture d'une valeur dans une variable, symbolisée par '←' ou '=' selon le langage.

Cas concret enseignant : *La métaphore de l'armoire d'archivage avec étiquettes : Imaginons la mémoire RAM comme un grand meuble à tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à spécifier la taille du tiroir (ex: 'Entier'). Affecter la valeur 150 dans 'solde_compte' revient à ouvrir le tiroir, jeter le papier qui s'y trouvait auparavant, et y déposer le nombre 150.*

2. Prise de Décision : Les Structures Conditionnelles (Si / Sinon)

Par défaut, un programme exécute les instructions de manière séquentielle, ligne après ligne. Pour introduire du comportement intelligent et adaptatif, on utilise la structure conditionnelle. Le programme évalue une condition logique (un prédicat booléen) : si la condition s'évalue à VRAI, le bloc d'instructions 'ALORS' est exécuté et le bloc 'SINON' est ignoré ; si elle s'évalue à FAUX, le bloc 'ALORS' est sauté et le bloc 'SINON' (si présent) est exécuté. Il est essentiel de maîtriser la table de vérité des opérateurs ET (exige que toutes les conditions soient vraies) et OU (nécessite au moins une condition vraie).

- Structure d'Alternative (SI ... ALORS ... SINON) : Branchement logique orientant le flux d'exécution selon la valeur de vérité d'une condition.
- Expression Booléenne : Expression logique dont le résultat vaut impérativement VRAI (True) ou FAUX (False).
- Opérateurs Logiques : Connecteurs (ET / AND, OU / OR, NON / NOT) permettant d'assembler des conditions complexes.

Cas concret enseignant : *Le cas du distributeur automatique de billets : Avant de délivrer l'argent, la machine teste la condition 'SI (solde_client >= montant_demandé) ET (billets_disponibles >= montant_demandé)'. Si la condition globale s'évalue à VRAI, le paiement est validé ; SINON, un message d'erreur s'affiche et la carte est restituée.*

3. Répétition et Automatisation : Les Structures Itératives (Boucles)

La puissance de calcul informatique réside dans la capacité à répéter des opérations des millions de fois sans erreur. La boucle POUR est privilégiée lorsqu'on connaît le nombre exact d'itérations à effectuer (par exemple, parcourir les 100 éléments d'un tableau). Elle gère automatiquement l'initialisation, le test d'arrêt et l'incrémentation d'un compteur. La boucle TANT QUE est utilisée lorsque le nombre d'itérations dépend d'un événement dynamique (ex: attendre une saisie valide de l'utilisateur). Attention : dans une boucle TANT QUE, si la condition d'arrêt ne devient jamais fausse, le programme entre dans une boucle infinie et plante.

- Boucle Pour (FOR) : Structure itérative bornée, utilisée lorsque le nombre de répétitions est connu à l'avance.
- Boucle TantQue (WHILE) : Structure itérative non bornée, répétée tant qu'une condition logique reste VRAIE.
- Variable d'Incrémentation / Compteur : Variable modifiée à chaque passage dans la boucle pour garantir la fin de l'itération.

Cas concret enseignant : *La métaphore de l'entraînement d'un athlète : Si l'entraîneur dit 'Fais 10 tours de piste' -> c'est une boucle POUR (le nombre 10 est fixe et défini). Si l'entraîneur dit 'Cours tant qu'il ne pleut pas' -> c'est une boucle TANT QUE (la fin dépend d'un test météo évalué à chaque tour).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' est-elle valide en programmation alors qu'elle est mathématiquement impossible ?
Réponse du professeur : Parce que le symbole '=' en programmation n'est pas une égalité mathématique, mais une instruction d'affectation. L'ordinateur évalue d'abord le côté droit (il prend la valeur actuelle stockée dans 'x' et lui ajoute 1), puis il range le résultat final dans la case mémoire 'x' située à gauche. On appelle cela une incrémentation.

Question étudiant : Que se passe-t-il si j'oublie de modifier la variable de contrôle à l'intérieur d'une boucle TantQue ?
Réponse du professeur : La condition restera toujours vraie, ce qui provoquera une boucle infinie. Le programme restera bloqué indéfiniment dans cette boucle jusqu'à ce que le système d'exploitation force son interruption.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme de tous les entiers de 1 jusqu'à N (1 + 2 + ... + N).

Résolution pas à pas :
Étape 1 : Déclarer les variables nécessaires : 'N' (Entier, valeur limite), 'i' (Entier, compteur de boucle), 'somme' (Entier, accumulateur). Étape 2 : Demander et lire la valeur de N saisie par l'utilisateur. Étape 3 : Initialiser l'accumulateur : somme ← 0. Étape 4 : Utiliser une boucle POUR allant de i = 1 jusqu'à N avec un pas de 1. Étape 5 : À chaque itération, exécuter l'affectation : somme ← somme + i. Étape 6 : Une fois la boucle terminée, afficher le contenu de la variable 'somme'.

Résultat : Si l'utilisateur saisit N = 5, les itérations successives calculent : somme = 0+1=1, puis 1+2=3, puis 3+3=6, puis 6+4=10, puis 10+5=15. Le programme affiche correctement le résultat : 15.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est utilisée lorsque le nombre d'itérations est connu d'avance (itératif borné), tandis que la boucle TANT QUE s'exécute tant qu'une condition logique est vraie sans connaître au départ le nombre exact de tours (itératif non borné).

Q : Que vaut la variable 'A' après l'exécution du bloc d'instructions : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : A vaut 8 (car 5 + 3 = 8) et B vaut 5 (car 8 - 3 = 5).

Q : Pourquoi est-il impératif d'initialiser une variable avant de lire sa valeur dans un calcul ?

R : Si une variable n'est pas initialisée, la case mémoire contient une valeur indéterminée (un 'déchet' binaire laissé par un autre programme), ce qui entraîne un comportement imprévisible ou une erreur d'exécution.